

The four tasks of software configuration management

A summary of Edward H. Bersoff's model (1984)

Claudio Gael Rodríguez Vega

Student ID: 221943339

Master's in Software Engineering

Centro Universitario de los Valles (CUValles), University of Guadalajara

1. Introduction

During the 1970s, a lot of software projects ran late and over budget. Software configuration management (SCM) was formalized in the following decade as a response. Systems kept getting bigger and the usual management methods couldn't keep up, because the work of coordinating a project grows much faster than the number of lines of code.

This also led to a broader idea of what software is. It was no longer just the executable program. It included the documentation and all the information needed to define, build, and maintain it.

Bersoff calls the goal of all this effort product integrity. A product with integrity meets the user's needs and the specified performance, is delivered on budget and on schedule, and can be traced at any point in its life cycle. Management has four product assurance disciplines to get there: configuration management, quality assurance (QA), verification and validation (V&V), and test and evaluation (T&E). SCM's job is to identify the system's configuration at specific points in time, so that changes can be controlled without losing traceability.

2. The four tasks

2.1. Configuration identification

This is where you decide which parts of the system go under control, how they're organized, and what label each version gets. The central concept is the baseline, a snapshot of the system at a given point in the life cycle. Later work starts from it. Bersoff compares the sequence of baselines to the frames of a movie strip: seen in order, they show how the system evolved.

The things being labeled are software configuration items (SCIs), arranged as a tree within each baseline. An SCI doesn't stay the same for the whole project. It starts as a specification, turns into a design document, and ends up as source code and an executable module. That trail lets both the developer and the buyer check that the product matches what was agreed.

2.2. Configuration control

This task deals with changes. Any modification to an SCI goes through a formal process in which someone proposes it, it gets evaluated, and in the end it's approved or rejected.

The core document is the engineering change proposal (ECP). It explains what should change and why, which SCIs and baselines are affected, and what it would cost in money and time. The decision belongs to the configuration control board (CCB), made up of representatives from the areas involved in the project. On top of that, written procedures govern how each request is handled. Developers make the technical change; SCM makes sure the process is followed.

Bersoff also mentions program support libraries (PSLs). These are repositories that store the approved versions of each component, restrict who can modify them, keep the change history, and can be used to rebuild documents.

2.3. Configuration status accounting

A change can take a long time to go from proposal to implementation. In the meantime, someone has to keep track of where the system stands compared with the official documentation. That's status accounting.

Bersoff lists nine things to record. In short: when each baseline and each SCI was created, what each SCI is, the status and content of each ECP, how the implementation of each change is going, the state of the documentation, and any deficiencies found in audits.

2.4. Configuration auditing

An audit compares the system against its requirements and baselines to see how closely they match. A new baseline isn't approved without one. It relies on technical reviews and on verification and validation (V&V) testing, and there are two sides to it. Verification checks that what was planned for each SCI in one baseline was actually achieved in the next. Validation asks a different question: whether that SCI solves the right problem, meaning whether it actually works for the user.

Audits give management visibility. They show things that would otherwise slip by, like requirements that got dropped along the way or features nobody asked for. They cost something, sure, but less than fixing a design flaw late in the project.

3. The risk of too much control

Bersoff points out a habit among people who work in product assurance: they tend to overcontrol. To explain it, he borrows an example from control theory. Picture carrying a cup of hot coffee, filled to the brim, from the kitchen to the table. If you keep your eyes on the cup the whole way, you're more likely to spill it. Every small wobble makes you correct, you overcorrect, you correct again, and the coffee ends up on the floor.

SCM can fall into the same trap. Bury the development team in paperwork and the project gets slower and more expensive. What Bersoff wants from SCM is visibility and control over the project, not a paralyzed development team. He argues for a product that is good enough, delivered on time and within budget, over a perfect one that never ships.

4. Conclusion

Each task covers something the others don't. Identification says what exists and what it's called, control filters changes, status accounting keeps a record of what happened, and auditing checks that the result is what was asked for. Drop one and traceability breaks.

To me, the most useful part of the paper is the warning about overcontrol. It's easy to read the four tasks as a checklist to apply as hard as possible, and Bersoff says the opposite: the amount of control has to fit the size of the project. Management needs to see what's going on, but without slowing down the team's technical work.

Reference

Bersoff, E. H. (1984). Elements of software configuration management. *IEEE Transactions on Software Engineering*, SE-10(1), 79-87.